EXECUTIVE SUMMARY

Target: http://multi-vuln.com

Scan ID: TORTURE-P2-262fae06

Scan Date: 2026-03-04 17:13:50

Scan Duration: N/A

Total Requests Sent: 29

Avg Latency: N/A

Peak Concurrency: N/A

Findings: 5 vulnerabilities detected

Severity Breakdown: Critical: 0 | High: 3 | Medium: 2 | Low: 0

AI Inference Calls: 0

LLM Avg Latency: N/A

Circuit Breaker Activations: 0

- High severity vulnerabilities identified in three categories pose significant security risks to system integrity and data confidentiality;
- Immediate action required includes patching all high-risk issues within 48 hours to mitigate potential exploitation threats;
- Long-term strategy should focus on implementing comprehensive vulnerability management processes for ongoing monitoring and remediation.

EXECUTIVE STRATEGIC IMPACT

{
"Strategic_Attack_Path_Analysis": [
"An adversary could exploit the XSS vulnerability in user input fields to inject malicious scripts, allowing them to execute arbitrary code on the vulnerable server and gain initial access.",
"By leveraging the SSRF vulnerabilities present across multiple services, attackers can manipulate internal network traffic, pivot between systems, and potentially escalate privileges by accessing sensitive data or executing unauthorized commands through compromised hosts.",
"The INFORMATION_DISCLOSURE vulnerability could be exploited to extract critical configuration details, user credentials, or other valuable information from vulnerable endpoints. This intelligence would enable the attacker to fine-tune their attack strategy, identify additional entry points, and ultimately achieve a high-impact objective such as full system compromise."
]
}

DETAILED FINDINGS

FILTER: INJECTION & FUZZING

Finding #1: Cross-Site Scripting (XSS)

HIGH

CWE: CWE-79

CVSS Score: 7.1 (HIGH)

THREAT SCORE: 71/100

Description:

- Vulnerability 'Cross-Site Scripting (XSS)' detected in target endpoint.

Impact:

- Unauthorized access to system resources confirmed.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'XSS' was intercepted to protect system integrity.

FORENSIC ANALYSIS

```
Method: GET | Param: param_0
URL: http://test-target-torture-.com/api/v1/endpoint_0
Analysis: The application failed to properly sanitize user input before rendering it in a web page, allowing malicious scripts to be injected and executed.
```

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.

Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

```
Vector Category: Cross-Site Scripting (XSS)
Raw Payload:     test_payload_0
Encoded:         746573745f7061796c6f61645f30
Encoding Type:   None
```

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

```
GET http://test-target-torture-.com/api/v1/endpoint_0 HTTP/1.1
Host: target
{}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

Remediation:

- Implement strict input validation.

Recommended Code Fix:

```
```python
from flask import Flask, request, render_template_string

app = Flask(__name__)

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        user_input = request.form['input']
        safe_output = escape(user_input)
        return render_template_string('<p>{{ safe_output }}</p>', safe_output=sa

    return render_template_string('''
        <form method="post">
            <input type="text" name="input" />
            <button type="submit">Submit</button>
        </form>
    ''')
```
```

Finding #2: AI Prompt Injection

CRITICAL

CWE: CWE-77

CVSS Score: 9.0 (CRITICAL)

THREAT SCORE: 90/100

Description:

- The endpoint is vulnerable to prompt injection attacks.
- User input can manipulate the response payload.
- No validation or sanitization of user-provided data.

Impact:

- Potential unauthorized access to sensitive information.
- Data tampering and manipulation by malicious actors.
- Denial-of-Service (DoS) through excessive requests.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'PROMPT_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

```
Method: PUT | Param: param_2
URL: http://test-target-torture-.com/api/v1/endpoint_2
Analysis: The application failed to properly validate and sanitize user input, allowing
malicious payload injection.
```

Table 1: Payload Decomposition

Component	Value	Technical Function
param_2	test_payload_2	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

```
Vector Category: AI Prompt Injection
Raw Payload:     test_payload_2
Encoded:         746573745f7061796c6f61645f32
Encoding Type:   None
```

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_2'
```

HTTP Traffic Snapshot:

Request:

```
PUT http://test-target-torture-.com/api/v1/endpoint_2 HTTP/1.1
Host: target
{}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

Remediation:

- Implement input validation and sanitization for all user-provided data.
- Use parameterized queries or prepared statements to prevent SQL injection.
- Enforce strict access controls and authentication mechanisms.

Recommended Code Fix:

```
// Example of secure code snippet using parameterized query
SELECT * FROM users WHERE id = ?;
```

FILTER: [INFORMATION_DISCLOSURE]

Finding #3: Information Disclosure

MEDIUM

CWE: CWE-200

CVSS Score: 6.3 (MEDIUM)

THREAT SCORE: 63/100

Description:

- The API endpoint /api/v1/endpoint_1 is returning sensitive data in the response payload.
- No authentication or authorization checks are enforced for this endpoint, allowing unauthorized access to confidential information.
- The exposed payload (test_payload_1) contains potentially privileged data that could be exploited by malicious actors.

Impact:

- Unauthorized disclosure of sensitive data can lead to privacy breaches and potential misuse of confidential information.
- Exposure of internal or proprietary data may compromise system integrity, security posture, and regulatory compliance.
- Attackers can leverage the disclosed payload for further reconnaissance, privilege escalation, or launching targeted attacks against other systems.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'INFORMATION_DISCLOSURE' was intercepted to protect system integrity.

FORENSIC ANALYSIS

```
Method: GET | Param: param_1
URL: http://test-target-torture-.com/api/v1/endpoint_1
Analysis: The application failed to properly validate and sanitize user input, allowing
untrusted data to be processed.
```

Table 1: Payload Decomposition

Component	Value	Technical Function

param_1	test_payload_1	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

```
Vector Category: Information Disclosure
Raw Payload:     test_payload_1
Encoded:         746573745f7061796c6f61645f31
Encoding Type:   None
```

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_1'
```

HTTP Traffic Snapshot:

Request:

```
GET http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1
Host: target
{}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

Remediation:

- Implement proper authentication mechanisms (e.g., API keys, JWT tokens) to restrict access to this endpoint.
- Validate and sanitize all incoming request payloads before processing them to prevent unauthorized data exposure.
- Apply least privilege principles by ensuring the service account or user executing this endpoint has only necessary permissions.

Recommended Code Fix:

```
// Example secure code snippet (Node.js/Express)
const express = require('express');
const app = express();

app.get('/api/v1/endpoint_1', authMiddleware, (req, res) => {
  // Validate and sanitize request payload before processing
  const data = req.body;
  // Process the sanitized data securely
  res.json({ message: 'Data processed successfully' });
```

```
});

function authMiddleware(req, res, next) {
  // Implement authentication logic here (e.g., API key validation)
  if (!req.headers['x-api-key']) {
    return res.status(401).json({ error: 'Unauthorized' });
  }
  next();
}

module.exports = app;
```

FILTER: SSRF

Finding #4: Server-Side Request Forgery

CRITICAL

CWE: CWE-918

CVSS Score: 9.6 (CRITICAL)

THREAT SCORE: 96/100

Description:

- The target URL http://test-target-torture-.com/api/v1/endpoint_3 is vulnerable to Server-Side Request Forgery (SSRF).
- An attacker could potentially manipulate the request payload 'test_payload_3' to force the server to make unauthorized requests.
- This vulnerability may lead to information disclosure, data exfiltration, or even remote code execution if exploited.

Impact:

- Unauthorized access to internal resources and sensitive data.
- Potential for data breaches and exposure of confidential information.
- Risk of service disruption through forced request redirections.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

```
Method: GET | Param: param_3
URL: http://test-target-torture-.com/api/v1/endpoint_3
Analysis: The application failed to validate and sanitize user input, allowing arbitrary
URLs to be injected into API requests.
```

Table 1: Payload Decomposition

Component	Value	Technical Function
param_3	test_payload_3	Direct reference to target resource.

Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

```
Vector Category: Server-Side Request Forgery
Raw Payload:     test_payload_3
Encoded:         746573745f7061796c6f61645f33
Encoding Type:   None
```

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_3'
```

HTTP Traffic Snapshot:

Request:

```
GET http://test-target-torture-.com/api/v1/endpoint_3 HTTP/1.1
Host: target
{}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

Remediation:

- Implement strict input validation and sanitization on all incoming requests, especially those containing user-provided payloads.
- Restrict outbound network access to only necessary services and limit the scope of allowed URLs in server responses.
- Deploy a Web Application Firewall (WAF) with rules specifically targeting SSRF patterns.

Recommended Code Fix:

```
// Example secure code snippet using Python's requests library
import requests, urllib.parse
def safe_request(url, payload):
    parsed_url = urllib.parse.urlparse(url)
    if not all([parsed_url.scheme, parsed_url.netloc]):
        raise ValueError('Invalid URL')
    request = requests.Request(method=parsed_url.method,
                               url=parsed_url.geturl(),
                               headers=payload)
    preprocessed_request = urllib.parse.urlencode(request.__dict__, doseq=True)
    response = requests.Session().send(preprocessed_request)
    return response.text
```

Finding #5: Server-Side Request Forgery

CRITICAL

CWE: CWE-918

CVSS Score: 9.6 (CRITICAL)

THREAT SCORE: 96/100

Description:

- Vulnerability 'Server-Side Request Forgery' detected in target endpoint.

Impact:

- Unauthorized access to system resources confirmed.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

```
Method: PUT | Param: param_4
URL: http://test-target-torture-.com/api/v1/endpoint_4
Analysis: The application failed to validate and sanitize user input, allowing arbitrary
URLs to be passed as parameters.
```

Table 1: Payload Decomposition

Component	Value	Technical Function
param_4	test_payload_4	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

```
Vector Category: Server-Side Request Forgery
Raw Payload:     test_payload_4
Encoded:         746573745f7061796c6f61645f34
Encoding Type:   None
```

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_4'
```

HTTP Traffic Snapshot:

Request:

```
PUT http://test-target-torture-.com/api/v1/endpoint_4 HTTP/1.1
Host: target
{}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

Remediation:

- Implement strict input validation.

Recommended Code Fix:

```python
from flask import request, jsonify

def sanitize_input(data):
    return {k: v for k, v in data.items() if not any(c.isalnum() or c == '_' for

@app.route('/api/endpoint', methods=['GET'])
def api_endpoint():
    user_data = request.args.get('data')
    sanitized_data = sanitize_input(user_data)
    return jsonify(sanitized_data), 200
```

SCAN TIMELINE

```
[Orchestrator] TARGET_ACQUIRED - 2026-03-04 17:13:50
[Sigma] JOB_ASSIGNED - 2026-03-04 17:13:50
[Alpha] LOG - 2026-03-04 17:13:50
[Beta] LOG - 2026-03-04 17:13:50
[Beta] LOG - 2026-03-04 17:13:50
[Alpha] LOG - 2026-03-04 17:13:51
[Gamma] LOG - 2026-03-04 17:13:51
[Alpha] LOG - 2026-03-04 17:13:51
[Gamma] LOG - 2026-03-04 17:13:51
[Alpha] LOG - 2026-03-04 17:13:51
[Kappa] LOG - 2026-03-04 17:13:51
[Beta] LOG - 2026-03-04 17:13:51
[Kappa] LOG - 2026-03-04 17:13:51
[Gamma] LOG - 2026-03-04 17:13:51
[Kappa] LOG - 2026-03-04 17:13:51
[Gamma] LOG - 2026-03-04 17:13:52
[Gamma] LOG - 2026-03-04 17:13:52
[Kappa] LOG - 2026-03-04 17:13:52
[Beta] LOG - 2026-03-04 17:13:52
[Kappa] LOG - 2026-03-04 17:13:52
[Gamma] LOG - 2026-03-04 17:13:52
[Gamma] LOG - 2026-03-04 17:13:52
[Gamma] VULN_CONFIRMED - 2026-03-04 17:13:52
[Gamma] VULN_CONFIRMED - 2026-03-04 17:13:52
[Gamma] VULN_CONFIRMED - 2026-03-04 17:13:53
[Gamma] VULN_CONFIRMED - 2026-03-04 17:13:53
[Gamma] VULN_CONFIRMED - 2026-03-04 17:13:54
[Gamma] VULN_CONFIRMED - 2026-03-04 17:13:54
[Kappa] GI5_LOG - 2026-03-04 17:14:00
```